

Side Effects & Data Fetching in React (Theory)

Welcome to Side Effects & Data Fetching in React! So far, your React components have managed internal state and rendered UI based on user interactions. In this session, you will learn how to handle **side effects** using the `useEffect` hook and how to communicate with external servers (like your future FastAPI backend) using asynchronous HTTP requests.

1. What is a Side Effect?

In React, rendering a component should ideally be a pure calculation—taking props and state and returning JSX. However, real-world apps often require actions that interact with the outside world. These are called **side effects**.

Examples of side effects include:

- Fetching data from an API (e.g., calling your FastAPI `/predict` endpoint).
- Directly modifying the browser DOM or setting up timers (`setTimeout` , `setInterval`).
- Subscribing to external event listeners or WebSocket connections.

2. The `useEffect` Hook

To manage side effects in functional components, React provides the `useEffect` hook.

A. Basic Syntax

```
import { useEffect, useState } from "react";

function DataFetcher() {
  useEffect(() => {
    // Code executed after the component renders
    console.log("Component rendered or updated!");
  }, []); // Dependency array
```

```
    return <div>Check console for effect logs.</div>;  
  }
```

B. Controlling Execution with the Dependency Array

The second argument passed to `useEffect` is the dependency array. It controls *when* the effect runs:

1. **No Dependency Array:** Runs after *every single* render (use with caution to avoid infinite loops).

```
useEffect(() => { /* runs on every render */ });
```

2. **Empty Dependency Array (`[]`):** Runs **only once** when the component first mounts (loads onto the screen). Ideal for initial data fetching.

```
useEffect(() => { /* runs once on mount */ }, []);
```

3. **Array with Variables (`[userId]`):** Runs on mount *and* whenever any specified variable changes value.

```
useEffect(() => { /* runs when userId changes */ }, [userId]);
```

3. Making HTTP Requests (`fetch` vs. `Axios`)

To connect your React frontend to your FastAPI backend, you need to make HTTP requests (GET, POST). JavaScript provides two primary tools for this:

A. The Native `fetch` API

Built into modern browsers, `fetch` allows you to make network requests using promises and `async/await`.

```
async function fetchData() {  
  try {  
    const response = await fetch("http://127.0.0.1:8000/api/data");  
    const data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.error("Error fetching data:", error);  
  }  
}
```

B. Axios Library

Axios is a popular third-party JavaScript library for HTTP requests. It offers cleaner syntax, automatic JSON transformation, and better error handling than native `fetch`.

- *Installation:* `npm install axios`
- *Example Usage:*

```
import axios from "axios";

const sendPredictionRequest = async (payload) => {
  try {
    const response = await axios.post("http://127.0.0.1:8000/predict",
payload);
    console.log("Prediction Result:", response.data);
  } catch (error) {
    console.error("API Error:", error);
  }
};
```

4. Reference Links & Further Reading

- **React useEffect Documentation:** Official guide on synchronizing components with external systems.
 - **MDN Using Fetch Guide:** Comprehensive overview of native browser requests.
 - **Axios GitHub Documentation:** Learn advanced request configuration, interceptors, and error handling.
-